50.041 Distributed Systems — Final Project Report

Project title: A Distributed Key-Value Store Backed by a Token-Ring Distributed Mutex

1. Introduction

For our project we set out to build a small but complete distributed key-value store that would let us exercise, end-to-end, the three pillars of the course: distributed synchronization, distributed resource management, and decentralized fault tolerance. Rather than stack our system on top of an existing coordination library, we wanted to re-derive each of these mechanisms from scratch so that the protocols we read about in lectures: Lamport clocks, token-ring mutual exclusion, the Bully algorithm, and Paxos would all appear explicitly in our source code.

The result is a cluster of Go processes that together expose a single `Get / Put / Delete` API. Writes are serialized by a token that circulates around a logical ring of nodes: only the holder is allowed to append to the replicated log, and every node replays the log as the token arrives, giving eventual consistency. On top of this we built a leader election for coordinator failure, a write-ahead log for crash recovery, and, as an optional stronger-consistency mode, a Paxos-style write path that bypasses the token. All these features are written by us and contain no drop-in library for the distributed logic.

2. System Overview

At a high level, every node in the cluster is identical. It listens on an HTTP/2 endpoint, serves three Connect-Go services (`KvService`, `TokenService`, `RingService`), and maintains a small amount of local state: an in-memory map, a Lamport clock, a pending-write queue, and a write-ahead log on disk. Nodes are arranged in a ring ordered by node ID, and a single "token" message circulates between them. The token carries a log of uncommitted write events and a Lamport timestamp, which together are enough to give us total ordering of writes, replication, and staleness detection.

Mapping the three project requirements onto our design:

- **Distributed synchronization** is provided by Lamport timestamps on the token plus global sequence numbers on log entries, and by a Bully-style leader election to pick a coordinator.
- **Distributed resource management** is the token ring itself, which acts as a distributed mutex gating all writes.
- **Decentralized fault tolerance** comes from three complementary pieces: the coordinator heartbeat, a token keep-alive goroutine, and a per-node write-ahead log that survives restarts.

Separately, we added a Paxos-inspired write mode as a stretch goal for decentralized consensus. Clients can toggle it at runtime to compare the DME-based and consensus-based write paths on the same cluster.

3. Design

3.1 Logical components

The codebase is split into a handful of small packages. `pkg/kv` owns the local store and the WAL, `pkg/node` owns the distributed logic, and `kv/v1/kv.proto` defines the wire format used for every RPC. We deliberately kept the layering thin: the KV layer has no idea that a ring exists, and the node layer treats the KV store as a black box that accepts `ApplyLog(events)`. This separation made it possible to unit-test the replication logic against an in-memory store and the persistence layer against a simple sequence of events.

The most important pieces are:

- A **Lamport clock** in `pkg/node/clock.go`, with a `RecvIfNewer` primitive we added so that a node can atomically reject an incoming message whose timestamp is older than its own.
- A **Token** value object in `pkg/node/token.go`. The token carries the holder ID, the global offset of the first log entry, the current minimum-applied sequence number across the cluster, a Lamport timestamp, and the uncommitted log itself.
- A **Registry** in `pkg/node/registry.go` that owns the peer list, the current coordinator, and the heartbeat / election goroutines.
- A **Node** in `pkg/node/node.go` that ties everything together and serves the public API.

3.2 Distributed synchronization

We use Lamport timestamps rather than real clocks because we only need a logical "happened-before" relation. Every time a node hands off the token it calls `Tick()` and stamps the new value into the token (`pkg/node/node.go:261`). Every time a node receives a token it calls `RecvIfNewer` on its clock (`pkg/node/node.go:292`); if the token's timestamp is strictly less than the node's own, the token is dropped as stale. This protects us against the case where a partition heals and an old token reappears after a fresh one has already begun circulating.

Log entries carry a second, independent ordering: each `KvEvent` is stamped with `Seq = LogOffset + index` when the token is serialized to protobuf (`pkg/node/token.go:43`). Because only the token holder ever appends to the log, this sequence is cluster-wide monotonic and gives us a total order of writes.

For coordinator election we implemented the Bully algorithm by the book. A node that suspects the coordinator is dead fires `ReceiveElectionRequest` at every peer with a higher ID (`pkg/node/registry.go:230`). If anyone answers within ten seconds, it backs off; otherwise it declares itself leader and broadcasts a new `SetPeers` that updates everyone's coordinator index. The new leader then spawns a heartbeat goroutine that pings every peer every ten seconds and evicts unresponsive ones.

3.3 Distributed mutex

The token ring is the heart of the system. We chose it over a centralized lock manager because it is decentralized by construction since there is no single node that all writes must go through. This gives us a natural place to piggyback the replicated log. Safety (at most one writer) follows from the uniqueness of the token; liveness follows from the ring topology (every node is visited in `O(N)` hops); and ordering is preserved because writes are only ever appended by the token holder.

Concretely, when a client issues a `Put`, the node does *not* touch its local map. It builds a `PendingEvent{Event, Done}` and pushes it into a buffered channel, then blocks on `Done` (`pkg/node/node.go:442-459`). The event is only flushed when this node next receives the token: the token-driven loop calls `drainPendingInto`, which appends every queued event to the token's log, applies them locally, and signals each `Done` channel. From the client's point of view the `Put` is synchronous; under the hood it is gated by the token.

Two optimizations were worth the code:

- **Log compaction.** If we never trimmed the log, the token's payload would grow indefinitely. We track `lastApplied` on every node and piggyback `MinApplied = min(everyone's lastApplied)` on the token. On every hop, entries with `seq < MinApplied` are discarded and `LogOffset` is advanced (`pkg/node/node.go:303-311`). In steady state the log on the wire stays small.
- **Idempotent replay.** A node that restarts (or skips a token because of a transient error) computes `startIdx = lastApplied - LogOffset` and applies only the slice it has not seen. We found this important in practice without it, a restarted node would double-apply writes and diverge from its peers.

To make the behaviour easy to observe in a demo, we added a `--manual` flag. In manual mode the automatic token-passing goroutine is disabled and the token only advances when someone issues `POST /tick`. The client CLI exposes this as a `tick` command, which is what we use to walk through the replication lag live during the demo.

3.4 Decentralized fault tolerance

Three mechanisms together are enough to keep the cluster alive through individual failures.

First, **crash recovery** is handled by the write-ahead log in `pkg/kv/kv.go`. `ApplyLog` appends every event to an append-only JSON file and `fsyncs` before returning (`pkg/kv/kv.go:122-153`). On boot, `OpenLog` calls `recover()` which replays the file into the in-memory map before the node starts accepting RPCs. A crashed-and-restarted node therefore re-joins with its state intact and can catch up on anything it missed via the token's replicated log.

Second, **coordinator failover** is provided by the election system described above. The registry runs a background goroutine (`pkg/node/registry.go:190-228`) that pings the coordinator once every ten seconds and flips to `CoordinatorUnreachable` on timeout, which in turn triggers `StartElection`.

Third, **the token itself has a keep-alive**. A dedicated goroutine in `pkg/node/node.go:117-131` watches the `freshestToken` reference once a minute. If it has not changed — which would indicate the token was lost to a

crashed successor — the last known holder re-sends it to its `Next()` peer. To avoid duplicate tokens at startup, a coordinator booting into a warm ring first asks every peer via `SeenToken` whether a token is already circulating, and only mints a new one if no peer has seen any (`pkg/node/node.go:144-184`).

3.5 Decentralized consensus (stretch goal)

We added a second write path, behind `POST /enable-paxos`, to get first-hand experience with the trade-offs between DME-based and consensus-based writes. When paxos mode is enabled, the elected coordinator runs a two-phase protocol:

1. **Prepare.** The coordinator broadcasts `ReceivePrepareProposalRound{round = r+1}` to all peers and waits for a majority of promises. Each peer accepts if the incoming round beats its stored one, with ties broken by node ID (`pkg/node/node.go:617-645`).
2. **Accept.** Every client `Put` is then wrapped in a `ReceiveProposalRoundChange` with a monotonically increasing `Seq`. Acceptors validate both the round and the expected sequence before applying locally (`pkg/node/node.go:647-675`).

This is not a production Paxos, it inherits the coordinator's availability and currently requires a full quorum rather than a strict majority, but it is enough to show the protocol working alongside the token-ring path on the same cluster.

4. Technical Challenges and Lessons Learned

A few design decisions took us more than one attempt.

The first was **where to put the client-facing synchronization boundary**. We tried two approaches before settling on the pending-event channel. Our first version had the `Put` handler append directly to the token if it happened to be held; this was simple but clients that arrived during token-less windows would see spurious failures. The current design (queue on the node, flush when the token arrives, block the client until the flush acknowledges) was both easier to reason about and gave us a natural place to drain on shutdown.

The second was **stale-token handling**. Early on we saw weird double-apply bugs when transient errors caused a token to be re-sent. Adding the Lamport timestamp to the token and the `RecvIfNewer` primitive made it explicit that "the older of two tokens never wins," which eliminated the entire class of bugs at once.

The third was **membership churn**. Re-broadcasting the peer list every time the coordinator changes means every node has to handle a `SetPeers` arriving mid-operation. Holding `registry.mutex` during both the heartbeat and the election required careful ordering with `tokenMu` to avoid deadlock. Documenting which lock is acquired first (registry, then token) was more valuable than any single code fix.

5. What has been implemented

- `Get / Put / Delete` KV RPCs with protobuf-validated input, backed by an in-memory map and an append-only JSON WAL with crash recovery.
- A token-ring distributed mutex with pending-event queueing, log replay, compaction via `MinApplied`, and stale-token rejection via Lamport timestamps.
- Dynamic ring membership through `Join` / `SetPeers`, plus bootstrap logic that detects whether a token is already circulating before minting a new one.
- A Bully leader election triggered by a coordinator heartbeat, with automatic peer eviction when a ping times out.
- A Paxos-style write mode that can be enabled or disabled at runtime.
- A REST control plane (`/tick`, `/status`, `/enable-paxos`, `/disable-paxos`) used by the CLI client and the demo.
- An interactive CLI client in `cmd/client/main.go`, a Dockerfile, and a `run.sh` script that generates a `docker-compose.yml` for an N-node cluster.
- Unit tests for the clock, registry, token, KV store, and WAL persistence.

6. How to Test and Deploy

6.1 Running the unit tests

```
go test ./pkg/kv/... ./pkg/node/...              # full suite
go test -race ./pkg/kv/... ./pkg/node/...        # with race detector
```

The tests cover the Lamport clock edge cases (`pkg/node/clock_test.go`), token protobuf round-tripping (`pkg/node/token_test.go`), ring parsing and `SetPeers` (`pkg/node/registry_test.go`), end-to-end token hops (`pkg/node/node_test.go`), and KV semantics + WAL recovery (`pkg/kv/kv_test.go`, `pkg/kv/persistence_test.go`).

6.2 Bringing up a cluster

The easiest path is Docker:

```
./run.sh 3 --manual       # 3-node cluster in manual-tick mode (demo-friendly)
./run.sh 5                # 5-node cluster with automatic token passing
docker compose down       # tear down
```

`run.sh` generates a `docker-compose.yml` that maps `node-i` to host port `8079 + i`, starts the cluster, and launches the CLI client already configured with every node's address.

To run without Docker, each node can be started directly:

```
go run . --id=node-1 --addr=:8080 \
  --ring="node-1=localhost:8080,node-2=localhost:8081,node-3=localhost:8082" \
  --manual
```

6.3 A guided demo session

The following transcript reproduces the behaviour we walk through in the final presentation and is the most direct way to see every feature in action:

```
> put mykey hello        # queued on node-1
> tick                   # node-1 flushes, passes the token to node-2
> get mykey
  node-1: hello
  node-2: (not found)    # node-2 has not replayed the log yet
> tick                   # node-2 replays the log
> get mykey
  node-1: hello
  node-2: hello
> tick                   # node-3 catches up
> enable-paxos           # switch to the consensus path
> put otherkey world     # replicated through prepare/accept instead of the ring
> status                 # inspect Lamport clocks, last_applied, coordinator
```

This exercise makes three things observable at once: the DME guarantee (only the token holder writes), the replication lag (visible between ticks), and the consensus path (same public API, different protocol underneath).

7. Repository Layout

```
.
├── main.go                  # node entry point
├── cmd/client/main.go       # interactive CLI
├── kv/v1/kv.proto           # wire protocol
├── pkg/
│   ├── kv/                  # local store + WAL
│   ├── node/                # node, registry, token, clock
│   └── gen/                 # generated protobuf / connect code
├── run.sh                   # docker-compose generator
├── Dockerfile
└── docker-compose.yml       # (generated)
```

8. Conclusion

Building this project from scratch forced us to confront the small details that lecture slides tend to gloss over: what exactly a “stale” token looks like on the wire, how the pending queue interacts with shutdown, what happens when two nodes simultaneously declare themselves leader. We think the final system is a faithful, non-trivial demonstration of the three required features. Synchronization via Lamport clocks and Bully election, resource management via the token ring, and fault tolerance via the WAL plus coordinator failover, and with Paxos mode, a decentralized consensus alternative that lives comfortably next to the DME path.